



Gymnázium J. K. Tyla

DarkFactory
Agentické a harnessové inženýrství:
Umělá inteligence v praxi

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Rozsah práce: 1623 slov / 11100 znaků
Rozsah práce: 3600 slov / 24399 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

[CZ] Tato odborná práce se zabývá principy agentického inženýrství (*agentic engineering*): efektivními inženýrskými praktikami pro vývoj pomocí umělé inteligence prostřednictvím agentických systémů a architekturou těchto systémů. Praktickým přínosem práce je návrh a implementace systému DarkFactory — agentního harnessu instalovatelného jako aplikace pro platformu GitHub (GitHub App). Systém usiluje o maximální možnou míru automatizace vývojového cyklu od interpretace požadavků v GitHub Issues, přes plánování, až po vývoj kódu a sloučení změn. Práce reflektuje, že současné agentní systémy nelze vnímat jako plně autonomní: jazykové modely vyžadují deterministické mantinely, správu kontextu a zapojení člověka (*Human-in-the-loop*).

Klíčová slova

Actions , , Agent Harness , Agent Loop , , , Container , , , DAG , Embedding , Flaky Test , , Guardrail , HITL , Hook , Issue , KV Cache , LLM , Loop Engineering , MCP , Plugins , Progressive Disclosure , , Pull Request , , Skills , , Squash , , Subagent , , , Transformer

Obsah

<u>Prohlášení</u>	2
<u>Anotace</u>	3
<u>Klíčová slova</u>	4
1 <u>Úvod</u>	7
1.1 <u>Motivace a vymezení problému</u>	7
1.1.1 <u>Úvod</u>	7
1.2 <u>Cíl práce a výzkumné otázky</u>	7
1.2.1 <u>Úvod</u>	7
1.2.2 <u>Hlavní cíl</u>	7
1.2.2.1 <u>Úvod</u>	7
1.2.3 <u>Dílčí cíle</u>	7
1.2.3.1 <u>Úvod</u>	8
1.2.4 <u>Výzkumné otázky</u>	8
1.2.4.1 <u>Úvod</u>	8
1.3 <u>Metodika práce</u>	8
1.3.1 <u>Úvod</u>	9
2 <u>Agentické AI</u>	9
2.1 <u>Vývojové prostředí a praxe</u>	9
2.1.1 <u>Úvod</u>	9
2.1.2 <u>Softwarové inženýrství</u>	9
2.1.2.1 <u>Úvod</u>	9
2.1.3 <u>Správa verzí</u>	9
2.1.3.1 <u>Úvod</u>	9
2.1.3.2 <u>Git</u>	9
2.1.3.2.1 <u>Úvod</u>	9
2.1.4 <u>Průběžná integrace</u>	12
2.1.4.1 <u>Úvod</u>	12
2.2 <u>Jazykové modely, chatboti a agenti</u>	13
2.2.1 <u>Úvod</u>	13
2.2.2 <u>Jazykový model</u>	14
2.2.2.1 <u>Úvod</u>	14
2.3 <u>Agentické inženýrství</u>	15
2.3.1 <u>Úvod</u>	15
2.3.2 <u>Promptové inženýrství</u>	15

2.3.2.1	<u>Úvod</u>	16
2.3.3	<u>Agentní harness</u>	16
2.3.3.1	<u>Úvod</u>	16
2.3.3.2	<u>Smyčka ReAct</u>	18
2.3.3.2.1	<u>Úvod</u>	18
2.3.3.3	<u>Vyvolávání nástrojů</u>	19
2.3.3.3.1	<u>Úvod</u>	19
2.3.3.4	<u>Dovednosti</u>	20
2.3.3.4.1	<u>Úvod</u>	20
2.3.3.5	<u>Kontextové inženýrství</u>	21
2.3.3.5.1	<u>Úvod</u>	21
2.3.3.6	<u>Inženýrství pracovních grafů</u>	22
2.3.3.6.1	<u>Úvod</u>	22
3	<u>DarkFactory: Architektura harnessu - Praktická část</u>	23
3.1	<u>Vývojové prostředí a praxe</u>	23
3.1.1	<u>Úvod</u>	23
3.1.2	<u>Správa verzí</u>	23
3.1.2.1	<u>Úvod</u>	23
3.1.2.2	<u>Git</u>	23
3.1.2.2.1	<u>Úvod</u>	23
4	<u>Výsledky a diskuse</u>	23
5	<u>Závěr</u>	23
	<u>Seznam obrázků a tabulek</u>	25
	<u>Seznam příloh</u>	26
	<u>Seznam zdrojů</u>	27

1 Úvod

1.1 Motivace a vymezení problému

V moderním softwarovém inženýrství () dosáhla automatizace vysokého stupně zralosti. Sestavení zdrojových kódů, běh testovacích sad, statická analýza i nasazování do produkce probíhají běžně bez nutnosti lidského zásahu. Hlavním úzkým hrdlem celého vývojového procesu tak zůstává samotná tvorba a modifikace zdrojového kódu — časová prodleva mezi zadáním nového požadavku v podobě úkolu či hlášení chyby a vytvořením otestované, bezpečně začlenitelné změny.

Inspirací pro překonání tohoto omezení je průmyslový koncept temné továrny (*Dark Factory*) — plně automatizovaného výrobního provozu, který funguje samostatně bez nutnosti stálé přítomnosti lidské obsluhy. Cílem tohoto přístupu není vytlačení lidského inženýra, nýbrž posun jeho role: člověk definuje záměr, architekturu a funkční specifikaci, zatímco mechanické, rutinní a opakující se úkony přebírají autonomní systémy.

Nástup velkých jazykových modelů (LLM) otevřel cestu k automatizaci syntézy kódu, avšak stávající nástroje vykazují zásadní limity. Většina současných řešení (konverzační asistenti a doplňování kódu v editoru) řeší pouze izolovaný krok v podobě návrhu textového fragmentu. Chybí jim hlubší integrace do vývojového cyklu repozitáře: přímá práce se souborovým systémem, schopnost interpretovat výstupy překladače, iterativně odstraňovat syntaktické regrese a respektovat deterministická pravidla projektu.

Ústřední inženýrská otázka této práce proto nespočívá v tom, zda jazykový model dokáže napsat fragment kódu. Zkoumáme, jaká kontrolní a dozorčí architektura — značovaná jako **Agent Harness** — musí model obklopot, aby bylo možné jeho výstupům v produkčním repozitáři spolehlivě důvěřovat a dosáhnout vysoké míry autonomie se zachováním lidského dohledu.

1.1.1 Úvod

1.2 Cíl práce a výzkumné otázky

1.2.1 Úvod

1.2.2 Hlavní cíl

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2.1 Úvod

1.2.3 Dílčí cíle

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).

- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (Human-in-the-loop), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3.1 Úvod

1.2.4 Výzkumné otázky

- VO1 (Míra automatizace a role člověka): Lze vývojový proces od zadání požadavku (GitHub Issue) po pull request strukturovat tak, aby role vývojáře spočívala výhradně v architektonickém dozoru a schvalování záměru (Human Gate), bez nutnosti ručního psaní rutinního kódu?
- VO2 (Řízení divergence a spolehlivost smyčky): Jakými architektonickými mechanismy lze v harnessu spolehlivě zabránit patologiím modelu (perseveraci, oscilaci a zacyklení v ReAct smyčce)?
- VO3 (Integrita paměti a eliminace sémantického posunu): Jak spravovat kontextové okno agenta při komplexních úlohách, aby nedocházelo k degradaci pozornosti (Context Rot) a ztrátě architektonických invariantů při kompresi?

1.2.4.1 Úvod

 **Hlubková kritika / Oponentura:** Oponentura k výzkumným otázkám: Otázka VO1 je formulována binárně („Lze vývojový proces strukturovat...“), což svádí k tautologické odpovědi. Rigorózní oponent bude žádat empirické vymezení: Jaké procento rutinních úloh (např. oprava chyby se selhávajícím testem vs. komplexní refaktoring) harness reálně odbaví bez ručního zásahu do kódu? Doporučujeme otázku v obhajobě doplnit o kritérium mezní složitosti úkolu a míry redukce kognitivní zátěže člověka.

1.3 Metodika práce

Práce má teoreticko-architektonický a inženýrský charakter. Vzhledem k dynamickému vývoji v oblasti autonomního softwarového vývoje práce důsledně zachovává a integruje zavedené anglické odborné názvy (např. *harness*, *pull request*, *agent loop*, *prompt engineering*, *skills* či *context rot*). Použití této terminologie je integrální součástí práce, neboť tyto anglické pojmy představují de facto celosvětové průmyslové standardy (*industry standards*), jejichž doslovný český překlad by byl nejednoznačný, zavádějící či v rozporu s běžnou inženýrskou praxí.

Postup práce sleduje strukturu inženýrského cyklu:

- 1. Analýza konceptu: Systematické zmapování limitů autoregresivních modelů, dynamiky kontextového okna, jevu Context Rot a rozhraní nástrojů.
- 2. Návrh architektury: Formulace modulárního modelu agent harnessu, správy stavu, exekučního pískoviště, bezpečnostních pojistek a orchestrace subagentů.

- 3. Kritické zhodnocení: Porovnání navržených principů s volnými agentními smyčkami a vymezení provozních limitů autonomního inženýrství.

1.3.1 Úvod

 **Strukturální upozornění:** Chybějící evaluační rámec v metodice: Metodika práce v současné podobě popisuje inženýrský postup, ale postrádá formální specifikaci evaluačního rámce: definici vzorku úloh pro ověření spolehlivosti (syntetické úlohy vs. reálné bugfixy), stanovení kontrolních metrik (úspěšnost na první pokus, spotřeba tokenů na úspěšný PR) a srovnávací baseline.

2 Agentické AI

Úvod

2.1 Vývojové prostředí a praxe

Soubor verzovacích, plánovacích, integračních a kontrolních postupů tvořících deterministické prostředí pro agentní vývoj softwaru. ⁽¹⁾

2.1.1 Úvod

GitHub poskytuje nad gitovým repozitářem koordinační vrstvu pro zadání práce, revizi změn a automatizaci. Tyto odpovědnosti jsou v práci dále rozloženy mezi samostatné koncepty Issue, Pull Request a GitHub Actions.

2.1.2 Softwarové inženýrství

Systematické uplatňování inženýrských principů na specifikaci, návrh, implementaci, ověřování, provoz a údržbu softwarových systémů. ⁽¹⁾

2.1.2.1 Úvod

2.1.3 Správa verzí

Správa a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a v případě potřeby vracet. ⁽²⁾

2.1.3.1 Úvod

2.1.3.2 Git

Distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře. ⁽²⁾

2.1.3.2.1 Úvod

Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem. Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém v kombinaci s platformou .

- Klíčové komponenty infrastruktury zahrnují:

- Distribuovaný systém Git⁽²⁾: Ukládá kompletní historii projektu v podobě jednotlivých revizí (commitů). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - ▶ Zadávání a sledování úkolů (Issues): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - ▶ Revize změn (Pull Requests): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - ▶ Automatizace (GitHub Actions): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

[+] Samotný : zde zůstává vymezen jako distribuovaný systém správy verzí. Hosting a koordinaci repozitáře popisuje : zadání práce **Issue** : revizní integraci **Pull Request** : a automatizaci **Actions** :

Základním bezpečnostním pravidlem při zapojení autonomních agentů do vývoje je striktní izolace rozpracovaného kódu. Stabilní kód v hlavní větvi (`main`) nesmí být nikdy přímo vystaven experimentům a chybám modelu. Agent proto veškeré úpravy provádí ve vyhrazených pracovních větvích odbočených ze základní linie projektu.

Tento princip přináší následující výhody:

- - **Ochrana produkční větve** [+] **Ochrana produkční větve**⁽²⁾: Hlavní větev (`main`) reprezentuje stabilní, otestovaný stav připravený k nasazení. Přímé zapisování do této větve je zakázáno jak lidským vývojářům, tak autonomním agentům.
- Dedikovaná větev pro každý úkol: Agent pro každé zadání dynamicky vytvoří novou samostatnou větev (např. `task/123-oprava-parseru` či `agent/feature-auth`).
- Izolace chyb a mezistavů: Případné syntaktické chyby, dočasné nefunkční stavy ani neúspěšné hypotézy neovlivňují stabilitu hlavní větve ani práci ostatních vývojářů v týmu.
- Bezpečné zahození nezdařených běhů: Pokud se agent dostane do slepé uličky nebo vyčerpá přidělený rozpočet kroků, celou větev lze smazat jedním příkazem bez jakýchkoliv následků pro zbytek repozitáře.

Pokud se hlavní větev během práce agenta posune dopředu v důsledku jiné aktivity v repozitáři, pracovní větev agenta se musí před dokončením zaktualizovat (`git rebase` nebo `git merge`), aby byla zajištěna bezkonfliktní integrace. Způsob, jakým se změny z pracovní větve začlení do větve hlavní, má zásadní dopad na dlouhodobou udržitelnost a čitelnost repozitáře. Autonomní agent při řešení úlohy obvykle postupuje iterativní metodou pokus-omyl: upraví soubor, spustí testy, odhalí překlep a provede další drobný commit. V pracovní větvi tak vzniká dlouhá sekvence pomocných a experimentálních záznamů.

Zatímco klasický merge commit přenese do hlavní větve veškeré dílčí commity a rebase je lineárně přeskládá, v agentním vývoji se jako optimální strategie uplatňuje - **Commit and Merge**: [+]Commit and Merge⁽²⁾:

- Sloučení mezikroků (Squash): Všechny commity z pracovní větve jsou spojeny do jediného nového commitu, který je vložen do main .
- Eliminace interního šumu: Pomocné commity vzniklé při ladění testů se do hlavní větve vůbec nedostanou; historie projektu zůstává čistá a přehledná podle pravidla: jeden úkol = jeden commit.
- Atomický návrat změn (git revert): Pokud se v budoucnu ukáže, že začleněná úprava zanesla do produkce nečekanou vadu, lze celý úkol vrátit jediným atomickým příkazem bez nutnosti rozplétat desítky dílčích mezikroků.

V agentním vývojovém procesu může Issue fungovat jako explicitní vstupní specifikace úlohy. Odděluje zadání od samotné implementace a poskytuje stabilní referenční bod pro plán, pull request i následnou revizi. - Pull request (PR) představuje stěžejní komunikační uzel mezi autonomním agentem a lidským inženýrem. Jedná se o formální žádost o začlenění navržených změn z pracovní větve do větve hlavní. V tomto bodě se plně uplatňuje princip zapojení člověka do smyčky (Human-in-the-loop): [+] Pull Request : — [cz]

Formální návrh na začlenění změn z jedné větve repozitáře do druhé, který slouží jako místo pro automatizované kontroly, lidskou revizi a diskusi nad navrženými úpravami.⁽²⁾.

V tomto bodě se plně uplatňuje princip HITL : agent kód samostatně navrhne a otestuje, avšak konečné rozhodnutí o jeho přijetí náleží vývojáři.

Rozhraní pull requestu integruje všechny podstatné informace na jednom místě:

- Řádkový diff: Vizualní srovnání původního a nového stavu, kde jsou jasné barevně odlišené přidané, změněné a smazané řádky.
- Strukturovaný souhrn změn: Agent v popisu PR srozumitelně shrne, jaké úpravy provedl, jakou logiku zvolil a na které původní issue reagoval.
- Výsledky automatických kontrol: Přehled stavu automatizovaných testů a linterů z GitHub Actions (zelený či červený indikátor).
- Revizní diskuse: Možnost vývojáře přidávat komentáře k libovolnému řádku kódu, klást doplňující dotazy nebo vyžadovat přepracování konkrétních částí.

Lidský vývojář v roli revizora (Reviewer) posuzuje celkový architektonický záměr a rozhoduje o schválení, vrácení k dopracování, či zamítnutí pull requestu. K tomu, aby byla kontinuální integrace efektivní, nestačí testy pouze spouštět — jejich úspěšné dokončení musí být systémově vnuceno.

- - Požadované kontroly (Required Checks): [+]Požadované kontroly (Required Checks)⁽³⁾:

Seznam úloh v GitHub Actions, které musí skončit explicitním úspěchem (zelený stav), aby bylo technicky možné pull request sloučit:

- Statická analýza a linter: Kontrola dodržení kódového stylu, odhalování mrtvého kódu a základních syntaktických prohřešků.

- ▶ Typová kontrola a build: Jistota, že kód lze bez chyb zkompileovat a že typový systém nezaznamenal nekonzistence.
 - ▶ Automatizované testy: Úspěšný průchod jednotkových i integračních testů ověřují-
cích požadované chování.
 - Pravidlo deterministického výsledku: Každá kontrola musí skončit jednoznačným vý-
sledkem; tiché přeskocení testu nebo nejednoznačný stav sloučení zablokuje.
- GitHub poskytuje pravidla ochrany větví - (*Branch Protection Rules*) [+](*Branch Protection Rules* ⁽²⁾), která zabraňují začlenění neověřeného kódu do stabilní větve `main`.
- Povinné schválení člověkem: Požadavek na explicitní autorizaci kódu lidským vývojářem
dříve, než GitHub povolí sloučení do produkční větve.

2.1.4 Průběžná integrace

Vývojová praxe, při níž se změny často integrují a automaticky ověřují sestavením, testy a dalšími kontrolami, aby se integrační chyby odhalily co nejdříve. ⁽³⁾

2.1.4.1 Úvod

Samotný jazykový model kód pouze generuje na základě statistických závislostí v tréno-
vacích datech; nemá schopnost vnitřně ověřit, zda je vytvořený program syntakticky
bezchybný a funkčně správný. Nezastupitelnou roli objektivního arbitra správnosti proto
plní kontinuální integrace (CI) ⁽³⁾.

- V rámci platformy GitHub zajišťuje kontinuální integraci automatizační platforma
Actions :

- Izolované běhové prostředí (Container): Workflow běží v deklarovaném runner pro-
středí se stanovenými nástroji a závislostmi, čímž se omezuje závislost na lokálním stavu
počítače vývojáře.
- Automatická exekuce: Integrační pipeline se automaticky spouští při každém pushi do
pracovní větve i při otevření pull requestu.
- Deterministická zpětná vazba pro agenta: Pokud překlad nebo testy selžou, chybový
protokol z terminálu je předán zpět do kontextu agenta, který na jeho základě provede
informovanou opravu kódu.

[+] Konkrétní automatizační platformu popisuje *Actions* ; izolaci běhu *Container* ; a
problematiku nedeterministických selhání *Flaky Test* . Koncept CI zde zůstává zaměřen na
integrační kontrakt a strojově ověřitelnou zpětnou vazbu.

-

 **Hlubková kritika / Oponentura:** Nestálost testů (Flaky Tests) v integračních bězích: Společně se na automatické testy v CI naráží na problém nestálých testů (*flaky tests*), které občas selžou kvůli časování, síťové odezvě či asynchronním stavům, aniž by kód obsahoval chybu. Pokud agent narazí na takto náhodně selhávající test, může začít nesmyslně upravovat správný kód ve snaze chybu odstranit. CI pipeline proto musí nestálé testy minimalizovat nebo umožnit automatické opakování selhaného běhu v čistém prostředí.

[+] Riziko nestálých testů je vyčleněno do samostatného konceptu *Flaky Test* . GitHub Actions spouští deklarovaná workflow v reakci na události repozitáře. V této práci představuje konkrétní automatizační prostředí, které realizuje CI kontroly a další repozitářové procesy. Kontejner může CI běhu poskytnout opakovatelné uživatelské prostředí s deklarovanými nástroji a závislostmi. Tím omezuje vliv lokální konfigurace vývojářského počítače, aniž by sám o sobě zaručoval úplnou bezpečnostní izolaci. Nestálé testy oslabují roli CI jako deterministické zpětné vazby. Agent může náhodné selhání mylně interpretovat jako regresi a začít měnit správný kód; proto je vhodné zdroje nedeterminismu omezovat a podezřelé běhy ověřovat opakováním v čistém prostředí.

2.2 Jazykové modely, chatboti a agenti

Konceptuální minimum o jazykových modelech a jejich kontextu potřebné pro pochopení agentních systémů. ⁽⁴⁾

2.2.1 Úvod

 **Metodické vymezení / Rozsah práce:** - Těžištěm této práce není strojové učení, matematická optimalizace vah ani trénování neuronových sítí. Jazykový model vnímáme jako hotovou inferenční komponentu vystupující v roli stochastického kognitivního jádra. Ústředním předmětem zkoumání je agentické inženýrství (*agent engineering*) a architektura agent harnessu pro autonomní vývoj softwaru. Následující text je proto záměrně zredukován na nezbytné konceptuální minimum potřebné pro pochopení kontextového okna, spotřeby tokenů, degradace pozornosti a rozhraní nástrojů. [+] Těžištěm této práce není strojové učení, matematická optimalizace vah ani trénování neuronových sítí. Jazykový model je zde uvažován jako hotová inferenční komponenta. Ústředním předmětem zkoumání je agentické inženýrství (*agent engineering*) a architektura agent harnessu pro autonomní vývoj softwaru. Teoretická část proto zachycuje pouze koncepty jazykových modelů, které jsou nutné pro popis agentního běhu, kontextu a nástrojových rozhraní.

- V agentickém softwarovém inženýrství vystupuje velký jazykový model (LLM) jako stochastické kognitivní jádro celého systému. Z hlediska vnitřní architektury se jedná o dekodovací transformer (*Decoder-only*; Transformer), jehož typickými představiteli jsou moderní modely řad Claude, GPT či DeepSeek ⁽⁴⁾. Role modelu nespočívá ve vystupování jako vševědoucí orákulum se spolehlivou znalostí okolního světa, nýbrž jako pokročilý generátor hypotéz, kódu a strukturovaných volání nástrojů řízený obdrženým kontextem.

Základní principy fungování modelu zahrnují:

- Autoregresivní predikce: Model zpracovává zadanou sekvenci textu a na jejím základě iterativně předpovídá nejpravděpodobnější následující symboly (tokeny).
- Stochastická povaha: Vzhledem k pravděpodobnostnímu vzorkování může model na totožný vstup reagovat mírně odlišně, což vyžaduje deterministické mantinely v nadřazeném agent harnessu.

Pro efektivní nasazení modelu do vývojového cyklu je nezbytné porozumět způsobu, jakým reprezentuje informace a jaké fyzické limity vymezují jeho operační paměť. [+] Základ modelové vrstvy je rozložen do samostatných konceptů: **LLM**, **Transformer**, a . Následující koncepty dále oddělují tokenizaci, reprezentaci dat, kontextové omezení a stav inferenčního běhu. — [cz] Systém založený na jazykovém modelu určený primárně k textové interakci s uživatelem; odpovídá na jednotlivé požadavky, ale sám o sobě nedisponuje autonomní prováděcí smyčkou ani nástroji pro samostatnou modifikaci okolního prostředí.. - : — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného inženýrského cíle.. [+] : — [cz] Softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně plánuje, vnímá stav prostředí a provádí vícezkrokové akce směřující k dosažení zadaného inženýrského cíle. ^(5, 6). Rozdíl mezi nimi nespočívá v odlišném jazykovém modelu, ale v architektuře jeho zapojení do pracovního prostředí.

Srovnání obou přístupů:

- Konverzační chatbot:
 - Reaguje pouze na přímé textové výzvy v uzavřeném okně chatu.
 - Nemá přímý přístup k souborovému systému ani k nástrojům operačního systému.
 - Uživatel musí navržený kód ručně zkopírovat, vložit do projektu a otestovat.
- Autonomní agent:
 - Je vybaven sadou výkonných nástrojů (*tools*) pro práci s repozitářem.
 - Aktivně prozkoumává soubory, modifikuje zdrojový kód, spouští testy a interpretuje jejich návratové kódy.
 - Funguje v autonomní prováděcí smyčce, v níž iterativně reaguje na reálnou odezvu vývojového prostředí.

2.2.2 Jazykový model

- Velký jazykový model je neuronový model trénovaný nad rozsáhlými textovými daty, který autoregresivně zpracovává a generuje posloupnosti tokenů. [+] Velký jazykový model je neuronový model založený na **Transformer**, trénovaný nad rozsáhlými textovými daty a generující posloupnosti tokenů prostřednictvím .

2.2.2.1 Úvod

Současné velké jazykové modely jsou typicky realizovány transformerovou architekturou. Pro generování textu se často používá dekodérové uspořádání, které nad dosavadní sekvencí vytváří reprezentace potřebné k odhadu následujícího tokenu. Autoregresivní generování probíhá iterativně: model zpracuje dosavadní posloupnost tokenů, vypočítá distribuci pravděpodobnosti následujícího tokenu a po jeho výběru celý krok opakuje nad rozšířenou posloupností. Pravděpodobnostní dekodování znamená, že shodný vstup nemusí vždy vytvořit totožný výstup. V agentním systému proto nelze provozní spolehlivost opřít pouze o model; kritická pravidla a stavové přechody musí vynucovat deterministická vrstva harnessu. Jazykový model nepracuje přímo se znaky ani slovy v lidském slova smyslu. Vstupní

text je nejprve deterministickým algoritmem převeden na číselné reprezentace, se kterými následně počítají maticové vrstvy neuronové sítě.

Tento proces zahrnuje následující pojmy:

- **Tokeny a tokenizér ()**: Token představuje základní diskrétní jednotku (celé slovo, slabiku či fragment znaků). Převod mezi textem a posloupností číselných tokenů zajišťuje tokenizér - (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE) [+] (nejčastěji na bázi algoritmu Byte Pair Encoding, BPE ⁽⁷⁾).
- **Embedding**: — [cz] Vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné sémantické vztahy mezi reprezentacemi. ⁽⁸⁾ (např. vektorová analogie král — muž + žena $\approx$ královna).
- **Jazyková asymetrie tokenizace**: Vzhledem k trénovacím datům optimalizovaným primárně pro angličtinu spotřebovávají flektivní jazyky s bohatou diakritikou (včetně češtiny) 2× až 3× více tokenů pro vyjádření téhož významu.

Z inženýrského hlediska je proto žádoucí vést systémové prompty, technické plány i komunikaci mezi nástroji v angličtině, aby se šetřila kapacita kontextu a snížila latence inference. Interakce mezi modelem, uživatelem a okolním vývojovým prostředím neprobíhá spojitě, nýbrž v diskrétních krocích označovaných jako tahy (*turns*). Každý tah představuje jednu ucelenou výměnu zprávy, na niž systém reaguje.

Životní cyklus tahů a správa paměti zahrnují:

- **Typy tahů v agentní smyčce**:
 - **Tah uživatele či prostředí (User Turn)**: Nové zadání úkolu nebo vnější událost.
 - **Tah modelu (Model Turn)**: Vygenerovaná odpověď nebo strukturovaný požadavek na spuštění nástroje.
 - **Tah nástroje (Tool Execution Turn)**: Zpětné hlášení výsledku exekuce (výpis souboru, výstup kompilátoru).
- **- Správa KV cache (KV Cache) [+] Správa KV cache (KV Cache ^(9,10))**: Aby inferenční engine nemusel při každém novém tahu přepočítávat celou historii od začátku, ukládá mezivýpočty klíčů a hodnot matic pozornosti do paměti.
- **- Kontextové okno () [+] Kontextové okno (⁽⁴⁾)**: Pevně limitovaná kapacita paměti modelu. Tento strop je dán hardwarovými limity GPU akceleratorů a kvadratickou složitostí plné pozornosti ($O(N^2)$ vzhledem k délce sekvence).

2.3 Agentické inženýrství

Inženýrská disciplína zaměřená na návrh, orchestraci a provoz agentických systémů kolem jazykových modelů, včetně nástrojů, kontextu, prováděcích smyček, bezpečnostních mantinelů a lidského dohledu. ⁽⁶⁾

2.3.1 Úvod

2.3.2 Promptové inženýrství

Inženýrská metodika systematického návrhu, strukturování a optimalizace instrukcí a systémových promptů pro řízení chování a mantinelů jazykového modelu. ⁽¹¹⁾

2.3.2.1 Úvod

- Základní chování agenta vymezuje systémový prompt ⁽¹¹⁾, který definuje jeho identitu, sadu dostupných nástrojů a provozní mantinely. [+] : slouží k systematickému návrhu instrukcí, které řídí chování agenta. Základní instrukce jsou obvykle součástí : — [cz] Instrukční vrstva s vysokou prioritou, která vymezuje roli, chování, dostupné prostředky a provozní mantinely jazykového modelu nebo agenta. ⁽¹¹⁾, který vymezuje roli agenta, dostupné nástroje a provozní mantinely. Při formulaci těchto pravidel však vývojáři narážejí na specifickou vlastnost autoregresivních modelů — problematické zpracování zákazů a negativních instrukcí.

Příčiny a inženýrská řešení tohoto jevu:

- Uskálí negativních instrukcí: Zákazy formulované negací (např. „nemazat existující testy“) modely často porušují, protože matice pozornosti (QK^T) asociativně aktivuje zakázaný pojem dříve, než autoregresní proces uplatní logický operátor negace.
- Afirmativní formulace: Pravidla je nutné formulovat pozitivně — namísto výčtu zákazů vymezit přesný postup a povolené mantinely chování.
- - Deterministická ochrana v harnessu: Kde nestačí prompt, musí zasáhnout kód agent harnessu — například zpřístupněním testovacích souborů pouze pro čtení nebo zablokováním destruktivních operací na úrovni systémového volání. [+] **Guardrail** : odděluje programově vynucené bezpečnostní a procesní podmínky od instrukcí, jejichž dodržení závisí na chování modelu.

2.3.3 Agentní harness

Agentní harness — aplikační a orchestrační vrstva obklopující inferenční jádro modelu, která zajišťuje běhové prostředí nástrojů, dynamickou správu kontextového okna, bezpečnostní mantinely, práci se stavem a deterministické řízení životního cyklu požadavku. ⁽¹²⁾

2.3.3.1 Úvod

Samotné inferenční jádro provádí výhradně matematické maticové operace nad zadanými váhami a vektory tokenů; orchestraci, práci se soubory, správu stavu a bezpečnostní mantinely zajišťuje Agent Harness .

Ústřední prováděcí funkcí, která v architektuře harnessu řídí iterativní koordinaci agenta v reálném vývojovém prostředí, je **Agent Loop** :. [cz] Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí.. Přímé spouštění kódu (*Code Execution*) umožňuje agentovi generovat skripty (bash, Python), které harness spouští v izolovaném terminálu. Tento model poskytuje maximální flexibilitu pro softwarový vývoj, avšak vyžaduje nekompromisní bezpečnostní izolaci.

 **Hlubková kritika / Oponentura:** Iluzorní bezpečnost pískoviště: Přímé spouštění netestovaného syntetického kódu v běžném Docker kontejneru nelze považovat za plnohodnotnou

bezpečnostní hranici (*security boundary*). Přístup k síti otevírá prostor pro útoky typu Server-Side Request Forgery (SSRF), úniky environmentálních tajností (GitHub tokeny, API klíče k LLM) přes skryté síťové kanály a kompromitaci CI infrastruktury. Pro bezpečný produkční provoz je nezbytná formální izolace na bázi microVM - (např. AWS Firecracker, gVisor) [+](např. AWS Firecracker⁽¹³⁾, gVisor) a striktní izolace síťových jmenných prostorů.

Pokud samotná instrukce v promptu neposkytuje dostatečnou záruku, harness může pravidlo vynutit deterministicky, například omezením přístupových práv, validací parametrů nástroje nebo odmítnutím nepovoleného stavového přechodu. Základním principem navrženého řešení není nekritická plná autonomie, nýbrž - **efektivní kooperace člověka a stroje** [+]efektivní kooperace člověka a stroje (HITL⁽¹⁴⁾). Autonomnímu systému náleží mechanické a rutinní úkony, zatímco klíčová architektonická a nevratná rozhodnutí zůstávají plně pod kontrolou vývojáře.

Řízení lidského dohledu staví na těchto pilířích:

- **Lidské schvalovací brány (*Human Gates*):** Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora:
 - 1. brána (Záměr a plán): Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód v souborech.
 - 2. brána (Sémantická revize): Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
- **Prevence únavy z revizí (*Review Fatigue*):** Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- **Dohledatelnost původního zadání:** Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- **Transparentnost selhání a deterministická eskalace:** Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

 **Hlubková kritika / Oponentura:** Kognitivní limity lidské schvalovací brány (*Review Fatigue*): Spoléhání se na finální sémantickou kontrolu diffu v pull requestu naráží na lidské kognitivní limity. Výzkumy prokazují, že u rozsáhlých diffů (nad 300–400 řádků) dramaticky klesá hloubka lidské pozornosti — vývojář kód pouze zběžně prohlédne a spoléhá na zelenou fajfku z CI. Aby byla lidská brána efektivní, harness musí diffy rozkládat do sémanticky sevřených mikrokroků, generovat interaktivní vysvětlení netriviálních rozhodnutí a explicitně zvýrazňovat změny v kritických architektonických komponentách.

Plugin rozšiřuje samotný harness programovým modulem. Na rozdíl od instrukční dovednosti tak může přidávat systémové adaptéry, ovladače nástrojů nebo deterministické zásahy přímo do běhové vrstvy.

2.3.3.2 Smyčka ReAct

Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (Reasoning + Acting), v němž model střídavě uvažuje, volá nástroje a vyhodnocuje pozorování z běhového prostředí. ⁽¹⁵⁾

2.3.3.2.1 Úvod

Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

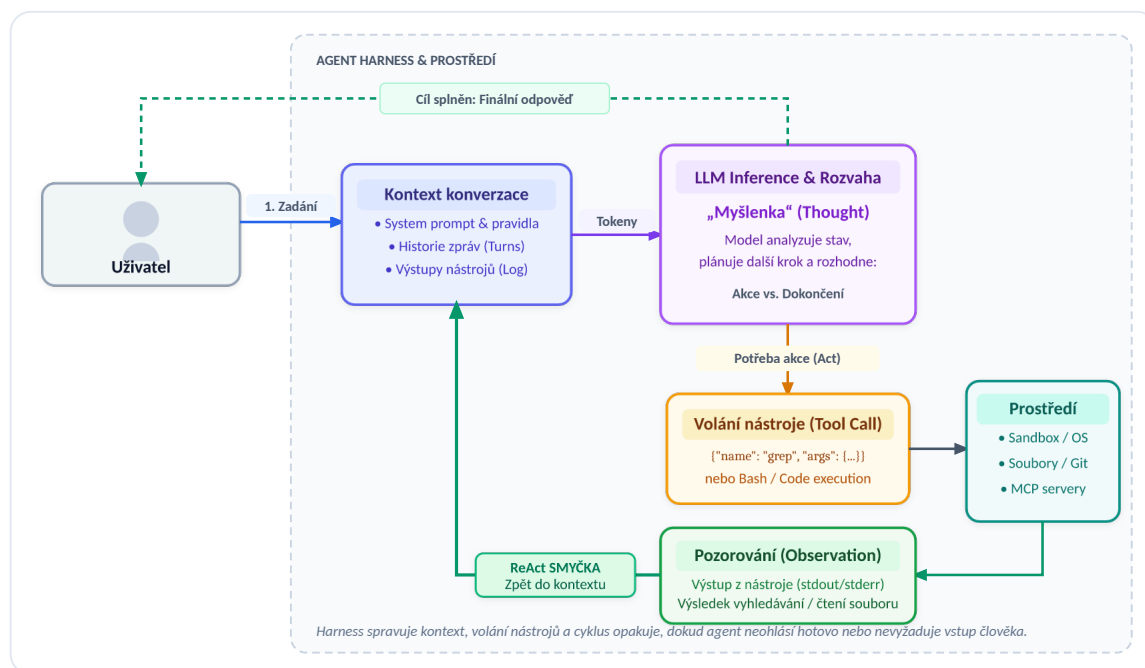
- V každé iteraci agentní smyčky harness zajišťuje tyto klíčové funkce:
- Inicializace a správa sezení: Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- Běhové prostředí nástrojů: Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.
- Řízení stavových přechodů a vynucování mantinelů (*Loop Engineering*): Dohled nad dodržováním procesních pravidel, detekce a zastavení uvíznutých běhů a vynucování lidských schvalovacích bran.

[+] Provozní odpovědnosti smyčky jsou odděleny do samostatných konceptů: *Tool Calling*, *Loop Engineering* a *Guardrail*. Samotná agentní smyčka zde popisuje jejich iterativní koordinaci.

Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*) ⁽¹⁵⁾, který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 1:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.



Obrázek 1: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Správa agentního sezení zahrnuje sestavení systémového promptu, načtení relevantního kontextu repozitáře, průběžné uchování stavu a sledování rozpočtů, například spotřeby tokenů nebo počtu iterací. Inženýrství prováděcí smyčky odděluje provozní řízení od samotného modelového rozhodování. Patří sem stavové přechody, podmínky ukončení, rozpočty běhu, opakování po selhání, detekce uvíznutí a pravidla pro eskalaci nebo lidské schválení.

- Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním. [+]Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním ⁽¹⁶⁾. V důsledku autoregresivní povahy se v kontextu snadno vytvoří pravděpodobnostní atraktor, který model uvězní v neproduktivním cyklu.

Mezi typické patologie patří:

- Perseverace a zacyklení: Opakované volání identického nástroje se stejnými neplatnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy.
- Oscilace a tékání (Thrashing): Střídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije modul A).
- Nekontrolovaná spotřeba zdrojů (Context Runaway): Rychlé vyčerpání kontextového okna i finančního rozpočtu na volání API bez dosažení cíle.

2.3.3.3 Vyvolávání nástrojů

Mechanismus, kterým model strukturovaně žádá harness o provedení externí akce nebo funkce s validovanými parametry. ⁽¹⁷⁾

2.3.3.3.1 Úvod

Aby mohl agent provádět reálné inženýrské operace, musí mu agent harness zpřístupnit systémové nástroje. Způsob, jakým jsou nástroje modelům předkládány, zásadně ovlivňuje ergonomii vývoje i bezpečnost celého systému.

- **Strukturované volání nástrojů (Tool / Function Calling)** [+]**Strukturované volání nástrojů (Tool / Function Calling** ⁽¹⁷⁾) používá vstupy a výstupy striktně validované vůči formálním JSON schémátům. Zajišťuje vysokou typovou bezpečnost, avšak přináší režii tokenů spotřebovaných na definice schémat. Pro sjednocení rozhraní mezi AI aplikacemi a externími nástroji či datovými zdroji vznikl otevřený standard **MCP**: — [cz] **Model Context Protocol** — otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC. ⁽¹⁸⁾ Namísto vytváření proprietárních rozhraní pro každou službu definuje MCP standardizovaný způsob komunikace.

Základní vlastnosti protokolu MCP:

- **Protokolové rozhraní:** Komunikace probíhá prostřednictvím standardu JSON-RPC (přes standardní vstup/výstup `stdio` nebo proud událostí `Server-Sent Events / SSE`).
- **Architektonické oddělení:** Implementace nástrojů běží jako samostatný proces mimo jádro harnessu. MCP servery fungují jako znovupoužitelné komponenty, které lze snadno sdílet napříč různými agenty a projekty.

2.3.3.4 Dovednosti

Znovupoužitelné modulární balíčky instrukcí (typicky definovaných v souboru SKILL.md), procedurálních pravidel a volitelných pomocných skriptů či zdrojů, které harness dynamicky načítá do kontextu agenta podle povahy řešeného úkolu. ⁽¹⁹⁾

2.3.3.4.1 Úvod

Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží - **Skills** -. [+] **Skills** : ⁽¹⁹⁾.

Architektura dovedností staví na následujících principech:

- **Definiční soubor SKILL.md** : Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.
- **Dynamické načítání pro úsporu kontextu**: Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Komplettní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- **Skripty () a záchytné body (Hook)**: Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

Kromě kontextových dovedností využívají pokročilé řídicí architektury také programové **Plugins** : — [cz] **Rozšíření běžící přímo v prostředí harnessu, která rozšiřují jeho exekuční jádro o specializované systémové adaptéry, ovladače nástrojů a deterministické záchytné body.** Zatímco *Skills* fungují jako kontextové procedury a instrukce interpretované modelem, pluginy rozšiřují samotný harness na nativní systémové úrovni. [+] **Načítání**

dovedností popisuje samostatný koncept **Progressive Disclosure**. Deterministické součásti dovedností jsou rozděleny mezi **Hook** a **Plugins**. U dovedností snižuje postupné zpřístupňování kontextovou režii: základní prompt obsahuje pouze přehled dostupných dovedností a úplný obsah příslušného `SKILL.md` se načte až tehdy, když jej agent pro konkrétní úlohu potřebuje. Skript poskytuje deterministickou exekuci pro úlohy, u nichž není vhodné znovu rozhodovat pomocí jazykového modelu, například pro opakovatelné transformace, validace nebo obslužné kroky dovedností. Hook váže deterministickou logiku na konkrétní událost životního cyklu harnessu, například před spuštěním nástroje, po dokončení kroku nebo při změně stavu běhu.

2.3.3.5 Kontextové inženýrství

Systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu, včetně instrukcí, paměti, nástrojových výsledků a externě načtených dat. ⁽²⁰⁾

2.3.3.5.1 Úvod

Správa aktivního kontextu je součástí kontextového inženýrství (). Při rozsáhlejších úlohách se kontextové okno nevyhnutelně zaplní. V okamžiku, kdy objem historie dosáhne kritické hranice, musí agent harness přistoupit ke kompakci kontextu - (**compaction**) [**+**] (**compaction** ⁽²¹⁾) — model je vyzván, aby dosavadní průběh sezení zkrátil do syntetického souhrnu, který nahradí starší část historie.

Tento proces však představuje destruktivní ztrátovou kompresi:

- Ztráta deterministických detailů: Model při rekurzivním zkracování vynechává přesná čísla řádků, signatury privátních funkcí, přesné cesty k souborům a doslovná chybová hlášení kompilátoru.
- Oslabení negativních pravidel: Explicitní zákazy (např. neměnit veřejné rozhraní API) bývají v souhrnu zveřejněny nebo zcela vypuštěny.
- Konfirmační zkreslení (*Confirmation Bias*): Model v souhrnu upřednostňuje fakta odpovídající jeho vnitřním statistickým asociacím na úkor netriviálních specifík konkrétního projektu.

Aby se předešlo ztrátě informací způsobené kompakcí, moderní agentní architektury přesouvají část paměti mimo samotné kontextové okno. Namísto spoléhání se na jediný lineární textový kontext se uplatňují strukturovaná externí úložiště.

K hlavním přístupům patří:

- - Hierarchická epizodická paměť (RAG) [**+**] Hierarchická epizodická paměť (RAG ⁽²²⁾) : Ukládání doslovných protokolů nástrojů a historie úloh do externí databáze; do kontextu se selektivně injektují pouze bezprostředně relevantní fragmenty.
- Persistentní graf stavu projektu (*Project State Graph*): Udržování explicitního, strukturovaného přehledu o stavu repozitáře (seznam modifikovaných souborů, otevřené úkoly, výsledky testů a platné invarianty) mimo kontextové okno.

Díky tomu může agent kdykoliv obnovit přesný stav projektu bez závislosti na ztrátovém rekurzivním shrnování. Schopnost jazykového modelu pracovat s dlouhým kontextem nelze posuzovat pouze podle nominální velikosti okna. Ačkoliv moderní modely deklarují kapacitu statisíců tokenů, jejich schopnost efektivně vyhledávat a logicky propojovat fakta s rostoucí délkou kontextu výrazně klesá. Tento jev se označuje jako — [cz] Degradace pozornosti a kvality logického uvažování modelu způsobená zaplněním kontextového okna dlouhou historií, šumem nebo vzájemně si konkurujícími informacemi, která vede k přehlížení instrukcí a ztrátě souvislostí..

V praxi se projevuje dvěma hlavními mechanismy:

- Lost in the Middle⁽²⁰⁾: Pozornostní vrstvy transformeru spolehlivě vnímají informace na samém začátku a konci okna, zatímco fakta umístěná uprostřed dlouhého textu jsou často přehlížena.
- Multi-Needle Reasoning: Schopnost logicky provázat několik na sobě závislých informací rozptýlených napříč různými soubory; s rostoucí délkou kontextu tato schopnost prudce klesá.

Při komplexním křížovém refaktoringu ve velkém kontextu proto model často přehlédne klíčové souvislosti, které by v menším a čistším okně zpracoval bez potíží. Opakovaná ztrátová komprese vede k závažné patologii známé jako sémantický posun (*Semantic Drift*). Pokud je historie sezení v dlouhém vývojovém běhu shrnována vícekrát po sobě, vzniká řetězec ztrátových transformací ($S_{k+1} = f(S_k, \Delta_k)$).

Rizika sémantického posunu spočívají v těchto jevech:

- Efekt tiché pošty: Drobné zkreslení či halucinace vzniklá v kole k je v kole $k + 1$ přijata jako nezpochybnitelný historický fakt.
- Divergence modelu od reality: Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému.

Výsledkem je stav, kdy agent sebevědomě reportuje vyřešení úkolu, ačkoliv reálný kód zůstává v nefunkčním či neúplném stavu.

2.3.3.6 Inženýrství pracovních grafů

Návrh agentních nebo automatizačních pracovních postupů jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky.⁽²³⁾

2.3.3.6.1 Úvod

Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

- Klíčové přístupy ke škálování zahrnují:

- Subagenti (*Subagents*⁽²³⁾): Hlavní orchestrátor dekomponuje rozsáhlou úlohu a deleguje dílčí kroky na specializované agenty (např. průzkumník repozitáře, plánovač, kódovací dělník). Po dokončení je kontext subagenta zahozen a orchestrátor obdrží pouze čistý výsledek, což chrání primární kontext před znečištěním (*context pollution*).

- Pracovní postupy jako grafy (DAG / Graph Engineering): Životní cyklus požadavku je modelován jako orientovaný acyklický graf (příjem → plán → kód → testy → schválení). Hrany definují striktní závislosti (needs); selhání v libovolném uzlu okamžitě zastaví navazující kroky.
- [+] Škálování je dále rozloženo na dva samostatné koncepty: hierarchickou delegaci prostřednictvím **Subagent** a explicitní závislosti pracovního postupu prostřednictvím **DAG**.
- Při hierarchické dělbě práce hlavní orchestrátor rozděljuje rozsáhlou úlohu a jednotlivé části deleguje specializovaným subagentům, například pro průzkum repozitáře, plánování nebo implementaci. Po dokončení dílčího běhu může nadřazený agent převzít pouze jeho výsledek namísto celé pracovní historie subagenta. V agentním workflow může DAG modelovat kroky jako uzly a jejich povinné závislosti jako hrany. Tím lze explicitně vyjádřit například posloupnost příjem požadavku → plán → implementace → testy → schválení a zabránit spuštění navazujícího kroku před splněním jeho předpokladů.

3 DarkFactory: Architektura harnessu - Praktická část

Úvod

3.1 Vývojové prostředí a praxe

Soubor verzovacích, plánovacích, integračních a kontrolních postupů tvořících deterministické prostředí pro agentní vývoj softwaru. ⁽¹⁾

3.1.1 Úvod

3.1.2 Správa verzí

Správa a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a v případě potřeby vracet. ⁽²⁾

3.1.2.1 Úvod

3.1.2.2 Git

Distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře. ⁽²⁾

3.1.2.2.1 Úvod

4 Výsledky a diskuse

5 Závěr

 **Metodické vymezení / Rozsah práce:** Poznámka k vypracování závěru: Závěr práce bude sepsán jako poslední krok po definitivním ucelení teoretických východisek agentického inženýrství a detailní architektury agent harnessu. Tato závěrečná kapitola syntetizuje zjištění o determinis-

tickém řízení autonomních agentů a zhodnotí formulované principy a výzkumné otázky bez vazby na dřívější ad-hoc testovací repozitáře.

Seznam obrázků a tabulek

Obrázek 1	<u>Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.</u>	19
-----------	---	----

Seznam příloh

Seznam zdrojů

1. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
2. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
3. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
4. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
5. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.
6. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
7. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
8. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
9. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
10. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
11. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
12. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>

13. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
14. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
15. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
16. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
17. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
18. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
19. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
20. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
21. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
22. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for

- Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
23. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryan W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
 24. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
 25. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
 26. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Proceedings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
 27. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.